

TALLER: FORMULARIOS AVANZADOS, VALIDACIONES Y PERSISTENCIA LOCAL EN SPA

Ecosistema React — Gestión Avanzada de Estados, LocalStorage y UI Avanzada con IA

INTRODUCCIÓN Y OBJETIVO DEL TALLER

En este taller darán el salto definitivo hacia la madurez en el desarrollo Frontend con React. Tomando como base la aplicación tienda-videojuegos con navegación SPA implementada anteriormente, expandiremos el formulario de registro para manejar estructuras de datos y tipos de componentes avanzados. Aprenderán a implementar validaciones reactivas en tiempo real para evitar datos corrompidos, añadirán persistencia de estado mediante LocalStorage y, de forma obligatoria, integrarán herramientas de Inteligencia Artificial para potenciar la maquetación CSS y el refinamiento de la experiencia de usuario (UX).

NUEVAS FUNCIONALIDADES A IMPLEMENTAR

- **Validación de Formularios Nativa y Dinámica:** Control exhaustivo de errores en el cliente antes de permitir el envío del formulario (bloqueo de espacios vacíos, longitudes mínimas y rangos de negocio).
- **Nuevos Tipos de Inputs y Componentes:** Incorporación de campos de fecha (date), áreas de texto extendidas (textarea) y entradas numéricas controladas (number).
- **Persistencia de Estado con LocalStorage:** Sincronización automatizada del estado de React con el almacenamiento interno del navegador para evitar pérdidas de datos al refrescar la página.
- **Estados de Feedback Visual Dinámico:** Renderizado de mensajes de error contextuales debajo de cada campo defectuoso y alertas de éxito temporales (Toasts).
- **Optimización Estética con Inteligencia Artificial (IA):** Uso guiado de prompts de IA para generar interfaces modernas, microinteracciones en componentes del formulario y paletas de colores coherentes.

PASO 1: NUEVOS CAMPOS EN EL MODELO DE DATOS Y FORMULARIO

Modificaremos el archivo FormularioVideojuego.jsx para recolectar información mucho más detallada de cada título. Debes añadir los siguientes componentes de formulario estructurados dentro de la maquetación de inputs existente:

- **Fecha de Lanzamiento (input type='date'):** Para registrar el día exacto en que salió al mercado. No se deben permitir de forma lógica fechas futuras a la fecha actual del sistema.
- **Sinopsis / Descripción (textarea):** Un cuadro de texto amplio para escribir una reseña corta del videojuego. Requisitos técnicos: mínimo 10 caracteres y máximo 250 caracteres.

- **Calificación de la Crítica (input type='number'):** Un campo numérico para ingresar la puntuación de Metacritic o de la tienda. El rango obligatorio del negocio debe situarse estrictamente entre 1 y 100.

PASO 2: SISTEMA DE VALIDACIONES Y CONTROL DE ERRORES

Para evitar que el usuario guarde videojuegos sin precio o con descripciones vacías que rompan la base de datos visual, implementaremos un estado local de errores en `FormularioVideojuego.jsx`.

Implementación de Validaciones Reactivas

```
// 1. Crear un estado llamado errores mediante un objeto vacío inicial:
const [errores, setErrores] = useState({});
```

```
// 2. Desarrollar una función validarFormulario() que verifique:
// - Que el nombre del videojuego no contenga solo espacios en blanco.
// - Que la calificación numérica esté estrictamente entre 1 y 100.
// - Que la sinopsis tenga al menos 10 caracteres.
```

Modifica el evento `handleSubmit`. Si la función de validación interna detecta fallos, se debe detener por completo el envío nativo del formulario (`e.preventDefault()`) y poblar el estado de errores para su renderizado:

```
if (Object.keys(erroresActivos).length > 0) {
  setErrores(erroresActivos);
  return;
}
```

Feedback Visual Dinámico: Debajo de cada input, añade un contenedor condicional de JSX que renderice el mensaje de error personalizado en color rojo únicamente si el campo es inválido:

```
{errores.nombre && <span className="error-mensaje">{errores.nombre}</span>}
```

PASO 3: PERSISTENCIA DE DATOS EN APP.JSX (LOCALSTORAGE)

Actualmente, si agregas o editas videojuegos y presionas F5 o recargas la pestaña del navegador, los datos regresan a su estado duro inicial. Solucionaremos este problema integrando `localStorage` mediante el ciclo de vida de React con el hook `useEffect`.

Reglas de Oro de Persistencia (src/App.jsx)

Lectura Inicial (Lazy State Initialization): Cambia el valor inicial del `useState` de tus videojuegos para que primero intente buscar de forma perezosa si ya existen datos guardados en la sesión del navegador. Si no hay nada, cargará el arreglo por defecto:

```
const [videojuegos, setVideojuegos] = useState(() => {
```

```
const datosGuardados = localStorage.getItem("lista_videojuegos");
return datosGuardados ? JSON.parse(datosGuardados) : [/* tus datos
iniciales */];
});
```

Escritura Automática (useEffect): Configura un efecto secundario que escuche atentamente cada vez que el estado videojuegos sufra una mutación (al agregar, editar o eliminar). Cuando detecte un cambio, guardará la nueva lista serializada:

```
useEffect(() => {
  localStorage.setItem("lista_videojuegos", JSON.stringify(videojuegos));
}, [videojuegos]);
```

🧠 PASO 4: COMPONENTE DE ALERTA DE ÉXITO (TOAST NOTIFICATION)

Crearemos un componente de notificación efímero y flotante para avisar al usuario que su acción en el CRUD fue procesada con éxito por el sistema.

- En src/components/, crea el archivo AlertaNotificacion.jsx junto a sus estilos estructurados en AlertaNotificacion.css.
- Este componente recibirá un prop llamado mensaje y se mostrará flotando con posición absoluta o fija en la esquina superior derecha de la pantalla con un fondo verde semi-transparente estético.
- Implementa un temporizador interno mediante setTimeout para que la alerta se desvanezca automáticamente tras transcurrir exactamente 3 segundos de haber sido montada en el DOM.

🛠️ PASO 5: OPTIMIZACIÓN Y MODIFICACIÓN DEL DISEÑO CON IA

Para cumplir con los estándares de interfaces modernas, utilizaremos un asistente de Inteligencia Artificial (ChatGPT, Gemini, Claude, etc.) para refinar estéticamente nuestro formulario y componentes. Debes aplicar de forma obligatoria las siguientes pautas:

Guía de Prompts y Modificaciones CSS con IA

Instrucciones técnicas para el alumno:

1. Copia tu estructura actual de clases del formulario y compártela con la IA usando el siguiente Prompt sugerido:

'Actúa como un Diseñador UX/UI Senior experto en React. Te proporciono mi estructura CSS de un formulario. Quiero que modifiques el diseño para que use una estética minimalista, bordes suaves, un área de texto (textarea) que se expanda elegantemente al hacer focus, inputs numéricos limpios y un layout responsivo. Devuélveme el CSS puro.'

2. Requisitos Visuales que la IA debe implementar en tu archivo FormularioVideojuego.css:

- Efectos de Focus Modernos: El borde de los inputs debe cambiar suavemente (usando transition: all 0.3s ease) hacia el color primario de la aplicación.
- Mensajes de Error Animados: Las clases de .error-mensaje deben aparecer con una pequeña

animación de entrada (opacity 0 a 1) y letras nítidas de advertencia.

- Diseño de Tarjetas Dinámicas: Modificar el contenedor de la tabla o cartas para que se acople perfectamente a las pantallas de dispositivos móviles.

PASO 6: VERIFICACIÓN Y REQUISITOS DE ENTREGA FINAL

Comprueba el correcto funcionamiento de tu ecosistema SPA en el navegador realizando la suite de pruebas del cliente:

- Intenta presionar 'Guardar' dejando el formulario completamente vacío. Comprueba que la aplicación no se rompa, se detenga el envío mediante JS y aparezcan todos los textos de validación en color rojo debajo de cada input correspondiente.
- Llena el formulario de manera correcta, incluyendo la fecha de lanzamiento, la sinopsis en el textarea y la nota de la crítica. Haz clic en guardar y verifica que la Alerta de Éxito (Toast) aparezca y desaparezca automáticamente a los 3 segundos.
- Recarga la página por completo presionando F5. Verifica en tu tabla que los videojuegos nuevos o editados sigan apareciendo intactos en la pantalla gracias a la lectura perezosa de LocalStorage.

CAPTURAS OBLIGATORIAS PARA EL ENVÍO

Para validar la correcta realización del taller, captura las siguientes 4 pantallas y envíalas a tu profesor por el canal oficial establecido:

- Captura del código de las validaciones: Bloque completo de la función `validarFormulario()` en el editor VS Code donde se aprecian las condiciones lógicas de negocio escritas por ti.
- Captura de la UI con errores visuales y diseño IA: La página del formulario mostrando los mensajes de error activados debajo de los inputs con los estilos refinados mediante Inteligencia Artificial.
- Captura de la consola de desarrollo (Pestaña Application): La pantalla de inspección de tu navegador abierta en Application -> Local Storage, demostrando que la clave 'lista_videojuegos' contiene el JSON estructurado con tus datos locales persistidos.
- Captura de la Alerta de Éxito en acción: Pantalla completa de la aplicación web mostrando la notificación flotante (Toast) verde tras guardar, editar o eliminar un videojuego.